

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	B V Chaitanya Reddy	Reg. No.:	192425376
Section / Batch:	2024	Date:	

Code of Conduct

I B V Chaitanya Reddy(192425376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __chaitanya_____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} | \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	3	3	4	3	3	16	81%
2	3	3	3	4	3	16	
3	4	3	4	3	3	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1 – E-Commerce Smart Search and Product Prediction System

Q1. MLE of Bigram Probability $P(\text{learning} \mid \text{machine})$

Formula:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Given:

- $C(\text{machine learning}) = 3$
- $C(\text{machine}) = 3$

Therefore,

$$\begin{aligned} P(\text{learning} \mid \text{machine}) &= \frac{C(\text{machine learning})}{C(\text{machine})} \\ &= \frac{3}{3} = 1.0 \end{aligned}$$

Final answer:

$$\boxed{P(\text{learning} \mid \text{machine}) = 1.0}$$

Interpretation:

The probability is 1.0 or 100%. This means that in the training corpus, every occurrence of "machine" is followed by "learning". Therefore, the e-commerce search system strongly predicts "learning" when the customer types "machine".

Q2. Backoff Model for "machine learning transforms"

The sequence "machine learning transforms" is unseen because:

$$C(\text{machine learning transforms}) = 0$$

Therefore, the trigram probability is:

$$P(t, \text{ransforms} \mid \text{machinelearning}) = 0$$

A backoff model moves to a lower-order model when the higher-order N-gram is unseen.

We can use:

$$P(\text{transforms} \mid \text{learning})$$

But the corpus contains:

- learning improves = 1

- learning enables = 1
- learning drives = 1

There is no occurrence of "learning transforms".

Therefore the bigram probability is also zero:

$$P(\text{transforms} \mid \text{learning}) = 0$$

The model then backs off to the unigram probability:

$$P(\text{transforms}) = \frac{C(\text{transforms})}{N}$$

Since "transforms" does not occur in the given corpus:

$$C(\text{transforms}) = 0$$

Therefore:

$$P(\text{transforms}) = 0$$

Final answer:

$$P(t, \text{ransforms} \mid \text{machinelearning}) \approx 0$$

Interpretation:

Backoff is useful because real-world search queries frequently contain unseen word combinations.

Instead of completely failing when a trigram is unseen, the model tries bigram and unigram probabilities. With a larger corpus or smoothing, an unseen word such as "transforms" could receive a small non-zero probability.

Q3. Deleted Interpolation probability of "machine learning improves"

Given:

$$\lambda_1 = 0.5$$

Trigram weight

$$\lambda_2 = 0.3$$

Bigram weight

$$\lambda_3 = 0.2$$

Unigram weight.

Formula:

$$P(w_i, | w_{i-2}w_{i-1}) = \lambda_1 P(w_i, | w_{i-2}w_{i-1}) + \lambda_2 P(w_i | w_{i-1}) + \lambda_3 P(w_i)$$

For "machine learning improves":

Step 1: Trigram probability

$$P(i, mproves | machinelearning) = \frac{C(machine\ learning\ improves)}{C(machine\ learning)}$$

Given:

$$C(machine\ learning\ improves) = 1$$

$$C(machine\ learning) = 3$$

Therefore:

$$P(i, mproves | machinelearning) = \frac{1}{3} = 0.3333$$

Step 2: Bigram probability

$$P(improves | learning) = \frac{C(learning\ improves)}{C(learning)}$$

The three occurrences of "machine learning" indicate:

$$C(learning) = 3$$

Therefore:

$$P(improves | learning) = \frac{1}{3} = 0.3333$$

Step 3: Unigram probability

The corpus has 15 words:

- machine = 3
- learning = 3
- improves = 1
- business = 1
- enables = 1
- automation = 1
- drives = 1
- innovation = 1

So:

$$P(improves) = \frac{1}{15} = 0.0667$$

Step 4: Interpolation

$$\begin{aligned} P &= 0.5(0.3333) + 0.3(0.3333) + 0.2(0.0667) \\ &= 0.16665 + 0.09999 + 0.01334 \\ &= 0.27998 \end{aligned}$$

Therefore:

$$P(i, mproves \mid machinelearning) \approx 0.28$$

Interpretation:

Deleted interpolation combines trigram, bigram and unigram information. The trigram captures detailed context, the bigram provides additional contextual information, and the unigram prevents the probability from becoming completely dependent on sparse higher-order N-grams.

Q4. Entropy of prediction distribution

Given:

$$P(improves) = 0.33$$

$$P(enables) = 0.33$$

$$P(drives) = 0.33$$

Entropy formula:

$$H = - \sum P(x) \log_2 P(x)$$

Therefore:

$$H = -[0.33 \log_2(0.33) + 0.33 \log_2(0.33) + 0.33 \log_2(0.33)]$$

$$H = -3(0.33 \log_2(0.33))$$

Since:

$$\log_2(0.33) \approx -1.599$$

$$H \approx 3(0.33)(1.599)$$

$$H \approx 1.583$$

Final answer:

$$H \approx 1.58 \text{ bits}$$

Interpretation:

The entropy is relatively high because all three words have almost equal probabilities. The model is uncertain about which word will occur next.

If one word had a probability close to 1, entropy would be low and prediction would be more certain.

Q5. Python Program – Case Study 1

This version is compact and suitable for Google Colab.

```
import nltk
import numpy as np
import math
from collections import Counter

# Download tokenizer
nltk.download('punkt')

# Corpus
text = """machine learning improves business,
machine learning enables automation,
machine learning drives innovation"""

# Tokenization
words = nltk.word_tokenize(text.lower())

# Generate n-grams
unigrams = Counter(words)
bigrams = Counter(zip(words[:-1], words[1:]))
trigrams = Counter(zip(words[:-2], words[1:-1], words[2:]))

N = len(words)

print("Number of words:", N)
print("\nUnigrams:", unigrams)
print("\nBigrams:", bigrams)
```

```
print("\nTrigrams:", trigrams)
```

```
# Q1 MLE Bigram
```

```
p_bigram = bigrams[("machine", "learning")] / unigrams["machine"]
```

```
print("\nQ1: P(learning | machine)")
```

```
print("= 3 / 3 =", p_bigram)
```

```
# Q2 Backoff
```

```
word = "transforms"
```

```
if trigrams[("machine", "learning", word)] > 0:
```

```
    p_backoff = trigrams[("machine", "learning", word)] /\
                bigrams[("machine", "learning")]
```

```
elif bigrams[("learning", word)] > 0:
```

```
    p_backoff = bigrams[("learning", word)] / unigrams["learning"]
```

```
else:
```

```
    p_backoff = unigrams[word] / N
```

```
print("\nQ2: Backoff probability")
```

```
print("P(transforms | machine, learning) =", p_backoff)
```

```
# Q3 Deleted Interpolation
```

```
p_tri = trigrams[("machine", "learning", "improves")] /\
        bigrams[("machine", "learning")]
```

```
p_bi = bigrams[("learning", "improves")] /\
        unigrams["learning"]
```

```
p_uni = unigrams["improves"] / N
```

```
l1, l2, l3 = 0.5, 0.3, 0.2
```

```
p_interp = l1*p_tri + l2*p_bi + l3*p_uni
```

```
print("\nQ3: Deleted Interpolation")
```

```
print("Trigram =", p_tri)
```

```
print("Bigram =", p_bi)
print("Unigram =", p_uni)
print("Final probability =", p_interp)
```

```
# Q4 Entropy
```

```
prediction = np.array([0.33, 0.33, 0.33])
```

```
entropy = -sum(p * math.log2(p) for p in prediction)
```

```
print("\nQ4: Entropy")
```

```
print("Entropy =", entropy, "bits")
```

```
# Next-word prediction
```

```
next_words = {
    "improves": 0.33,
    "enables": 0.33,
    "drives": 0.33
}
```

```
prediction_word = max(next_words, key=next_words.get)
```

```
print("\nFinal next-word prediction:", prediction_word)
```

CASE STUDY 2 – AI-Powered Hospital Appointment Chatbot

Q1. Penn Treebank POS tagging

Sentence 1

"The sentence is:

Book an appointment with the doctor."

Word	POS Tag	Meaning
Book	VB	Verb
an	DT	Determiner
appointment	NN	Noun
with	IN	Preposition
the	DT	Determiner
doctor	NN	Noun
.	.	Punctuation

Therefore:

Book/VB an/DT appointment/NN with/IN the/DT doctor/NN

Here, "Book" means to reserve an appointment, so it is a verb.

Sentence 2

"The sentence is:

The book contains medical information."

Word	POS Tag	Meaning
The	DT	Determiner
book	NN	Noun
contains	VBZ	Verb
medical	JJ	Adjective
information	NN	Noun
.	.	Punctuation

Therefore:

The/DT book/NN contains/VBZ medical/JJ information/NN

Here, "book" refers to a physical/document object, so it is a noun.

Justification

The word "book" is ambiguous.

- "Book an appointment" → action → VB

- "The book contains..." $\rightarrow$ object $\rightarrow$ NN

Thus, context determines the POS tag.

Q2. HMM Probability Calculation

Given:

$$P(\text{book} \mid VB) = 0.7$$

$$P(\text{book} \mid NN) = 0.3$$

Transition/start probabilities:

$$P(\text{Start} \rightarrow VB) = 0.6$$

$$P(\text{Start} \rightarrow NN) = 0.4$$

For VB:

$$\begin{aligned} P(V, B \mid \text{book}) &= P(VB \mid \text{Start}) \times P(\text{book} \mid VB) \\ &= 0.6 \times 0.7 \\ &= 0.42 \end{aligned}$$

For NN:

$$\begin{aligned} P(N, N \mid \text{book}) &= P(NN \mid \text{Start}) \times P(\text{book} \mid NN) \\ &= 0.4 \times 0.3 \\ &= 0.12 \end{aligned}$$

Therefore:

$$P(b, ook \mid VB) = 0.42$$

$$P(b, ook \mid NN) = 0.12$$

Since:

$$0.42 > 0.12$$

the HMM chooses:

$$VB$$

Interpretation:

At the beginning of a sentence, the given start probability and emission probability together make VB more likely than NN.

Q3. Rule-Based vs Stochastic HMM Tagging

Feature	Rule-Based Tagging	HMM Tagging
Method	Hand-written linguistic rules	Probabilistic model
Context	Uses grammatical rules	Uses transition and emission probabilities
Training	Usually no statistical training	Requires corpus/training data
Ambiguity	Depends on quality of rules	Handles ambiguity probabilistically
Example	"Book an..." → VB	Calculates probability of VB and NN
Advantage	Easy to understand	More flexible with context
Limitation	Rules can become complex	Depends on training data

Healthcare chatbot

Rule-based tagging can provide predictable behavior for common medical expressions.

HMM tagging can handle ambiguous words using statistical evidence from language data.

Therefore, a hybrid approach can be useful for a healthcare chatbot.

Q4. Importance of Penn Treebank Tagset

The Penn Treebank tagset provides standardized POS labels such as:

- NN → noun
- VB → verb
- JJ → adjective
- DT → determiner
- IN → preposition
- VBZ → third-person singular verb

It supports downstream NLP tasks in the following ways:

1. Intent Detection

POS information helps distinguish actions.

Example:

"Book an appointment"

"Book" as VB indicates an action/request.

2. Entity Identification

Nouns can help identify important entities such as:

- doctor
- hospital
- appointment
- medicine

3. Response Generation

Correct syntactic information helps the chatbot understand sentence structure and generate appropriate responses.

4. Ambiguity Resolution

"book" can be VB or NN depending on context.

Therefore, standardized POS tagging improves the consistency of healthcare chatbot processing.

Q5. Python Program – Case Study 2

```
import nltk

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')

sentences = [
    "Book an appointment with the doctor.",
    "The book contains medical information."
]

print("TOKENIZATION AND POS TAGGING\n")

for sentence in sentences:
    tokens = nltk.word_tokenize(sentence)
    tags = nltk.pos_tag(tokens)

    print("Sentence:", sentence)
    print("Tokens:", tokens)
    print("POS Tags:", tags)
    print()

# HMM calculation for "book"

emission_VB = 0.7
emission_NN = 0.3

start_VB = 0.6
start_NN = 0.4
```

prob_VB = start_VB * emission_VB

prob_NN = start_NN * emission_NN

print("HMM CALCULATION")

print("P(book, VB) =", start_VB, "*", emission_VB,
 "=", prob_VB)

print("P(book, NN) =", start_NN, "*", emission_NN,
 "=", prob_NN)

if prob_VB > prob_NN:

print("Most probable tag: VB")

else:

print("Most probable tag: NN")

Expected HMM result:

P(book, VB) = 0.42

P(book, NN) = 0.12

Most probable tag: VB

Note: NLTK's automatic POS tagger may occasionally produce a different tag depending on its trained model. The HMM calculation above uses the probabilities explicitly given in the question.

CASE STUDY 3 – Financial News POS Tag Correction and Uncertainty Analysis

Q1. Apply Transformation-Based Rule

Initial tags:

market/NN

growth/NN

drives/NNS

investment/NN

Rule:

Change NNS to VBZ if the preceding word is tagged as NN.

For "drives":

- drives = NNS
- preceding word = growth
- growth = NN

Therefore the rule applies.

NNS → VBZ

Corrected sequence:

market/NN

growth/NN

drives/VBZ

investment/NN

Final answer:

<i>market/NN growth/NN drives/VBZ investment/NN</i>

Interpretation:

The rule correctly changes "drives" from plural noun (NNS) to third-person singular present verb (VBZ).

Q2. Why "drives" should be VBZ

Sentence:

"Market growth drives investment."

Grammatical structure:

Market growth → Subject

drives → Verb

investment → Object

"Market growth" is a singular noun phrase.

Therefore, the verb must agree with the singular subject:

Market growth + drives

"Drives" is a third-person singular present-tense verb.

Hence:

<i>drives/VBZ</i>

The surrounding context helps identify the correct tag because:

- preceding phrase forms a noun phrase
- "drives" follows the subject
- "investment" follows "drives" as its object

Thus the contextual rule correctly resolves the tagging error.

Q3. Relative Frequency / Probability

Given frequencies:

Word	Frequency
market	500
growth	350
drives	180
investment	420

Total:

$$500 + 350 + 180 + 420 = 1450$$

Probability formula:

$$P(\text{word}) = \frac{\text{frequency}(\text{word})}{\text{total frequency}}$$

Market

$$\begin{aligned} P(\text{market}) &= \frac{500}{1450} \\ &= 0.3448 \end{aligned}$$

Growth

$$\begin{aligned} P(\text{growth}) &= \frac{350}{1450} \\ &= 0.2414 \end{aligned}$$

Drives

$$\begin{aligned} P(\text{drives}) &= \frac{180}{1450} \\ &= 0.1241 \end{aligned}$$

Investment

$$\begin{aligned} P(\text{investment}) &= \frac{420}{1450} \\ &= 0.2897 \end{aligned}$$

Final table:

Word	Frequency	Probability
market	500	0.3448
growth	350	0.2414
drives	180	0.1241
investment	420	0.2897

The probabilities approximately sum to:

$$0.3448 + 0.2414 + 0.1241 + 0.2897 = 1$$

Interpretation:

Corpus frequency provides statistical evidence about word usage. High-frequency words have higher probability and can contribute to probabilistic language models and POS-tagging decisions.

Q4. Entropy of Word-Frequency Distribution

Entropy formula:

$$H = - \sum P(x) \log_2 P(x)$$

Probabilities:

- market = 0.3448
- growth = 0.2414
- drives = 0.1241
- investment = 0.2897

Therefore:

$$H = -[0.3448 \log_2(0.3448) + 0.2414 \log_2(0.2414) + 0.1241 \log_2(0.1241) + 0.2897 \log_2(0.2897)]$$

Approximate result:

$$H \approx 1.92 \text{ bits}$$

Important interpretation

The transformation rule changes the POS tag of "drives" from NNS to VBZ, but it does **not change the word frequencies**.

Therefore, if "word-frequency distribution" literally means the given distribution of the four words:

$$H_{\text{before}} = H_{\text{after}} \approx 1.92 \text{ bits}$$

The entropy remains unchanged because the transformation modifies the tag, not the word-frequency

counts.

This is an important point to mention in the exam.

If instead entropy were calculated over a separate distribution of POS tags before and after transformation, then the entropy could change. But that POS-tag frequency distribution is not provided in the question.

Q5. Python Program – Case Study 3

```
import nltk
import math
from collections import Counter

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')

sentence = "Market growth drives investment."

# Tokenization
tokens = nltk.word_tokenize(sentence)

# Given initial tags
initial_tags = [
    ('market', 'NN'),
    ('growth', 'NN'),
    ('drives', 'NNS'),
    ('investment', 'NN')
]

print("INITIAL POS TAGS")
print(initial_tags)

# Transformation-Based Tagger
corrected_tags = initial_tags.copy()

for i in range(1, len(corrected_tags)):
    word, tag = corrected_tags[i]
    prev_word, prev_tag = corrected_tags[i - 1]
```

```

if tag == "NNS" and prev_tag == "NN":
    corrected_tags[i] = (word, "VBZ")

print("\nCORRECTED POS TAGS")
print(corrected_tags)

# Corpus frequencies
freq = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}

total = sum(freq.values())

print("\nWORD FREQUENCY DISTRIBUTION")

probabilities = {}

for word, count in freq.items():
    p = count / total
    probabilities[word] = p
    print(word, ":", count, "Probability =", round(p, 4))

# Entropy
entropy = 0

for p in probabilities.values():
    entropy -= p * math.log2(p)

print("\nENTROPY")
print("Before transformation =", round(entropy, 4), "bits")

# Transformation does not change word frequencies

```

```
entropy_after = entropy
```

```
print("After transformation =", round(entropy_after, 4), "bits")
```

```
print("\nFINAL POS TAGGING RESULT")
```

```
for word, tag in corrected_tags:
```

```
    print(word + "/" + tag)
```

Expected final POS result:

market/NN

growth/NN

drives/VBZ

investment/NN

Expected entropy:

Before transformation = 1.9197 bits

After transformation = 1.9197 bits